

Common Pitfalls and Anti-Patterns

Most anti-patterns are earlier shortcuts catching up with you.

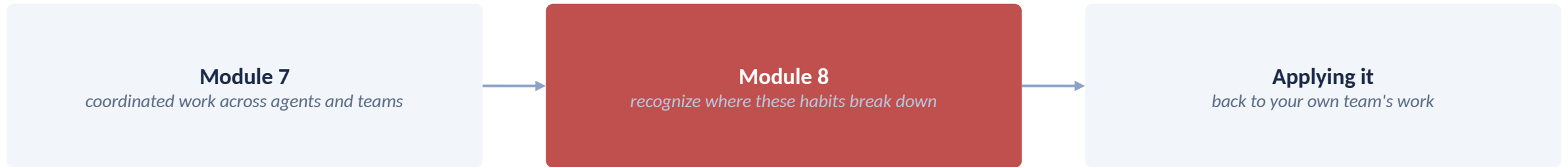
What You'll Learn

1 Recognize over-specifying and under-specifying, and find the right level of detail

2 Weigh token and cost tradeoffs against the rework they avoid

3 Learn from documented, framework-specific pitfalls in both tools

Every Habit From This Course Can Still Be Done Poorly



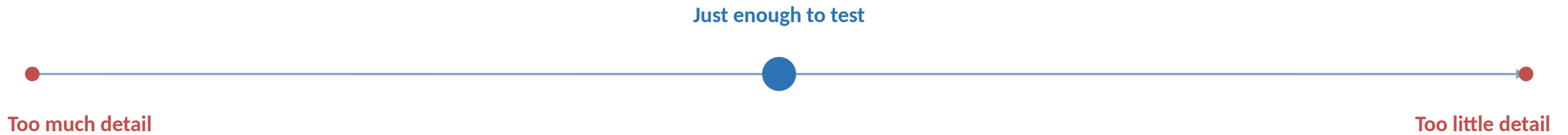
Seven modules of practices, checklists, and gates don't prevent every mistake, they just change what the mistakes look like. This module closes the course by naming the ones that show up most often.

PART 1 • THE BALANCE

Over- and Under-Specifying

Both directions push cost somewhere, just at different points in the lifecycle

Over- and Under-Specifying



Both directions push cost somewhere, just at different points in the lifecycle.

What Each Direction Actually Costs

Over-specifying

Locking down implementation details that belong in the plan, not the spec, makes the spec brittle: a minor implementation change now requires a spec edit and a re-review, even when intent hasn't changed.

Under-specifying

Leaving edge cases, defaults, or error behavior unstated hands the agent a gap to fill with a guess, exactly the 'weak assumption' failure mode from Module 6, discovered only after code exists.

The tell

A requirement is specified well when a reviewer could write a checklist item or a test against it without having to guess intent. That's the practical version of 'just enough to test.'

CHECKPOINT

Checkpoint: How do you know a requirement is specified at the right level?

- A reviewer could write a checklist item or a test against it without guessing intent.
- Over-specifying pushes cost into rigidity and rework on the spec itself.
- Under-specifying pushes cost downstream, into weak assumptions an agent has to guess at.

PART 2 • REVISITED

Token and Cost Tradeoffs

Treat it as a tradeoff to manage, not a cost to minimize blindly

Token and Cost Tradeoffs



Treat it as a tradeoff to manage, not a cost to minimize blindly.

WHY IT MATTERS

Most Anti-Patterns Are Visible in Hindsight, Rarely in the Moment



Checkpoint: Why does skipping a gate rarely feel costly in the moment?

- The cost of a skipped gate is deferred, it surfaces downstream, not at the point of the shortcut.
- More tokens spent on a clear spec and plan usually offsets against fewer rework cycles later.
- Minimizing token cost as a goal in itself tends to move the cost, not remove it.

PART 3 • NAMED AND DOCUMENTED

Framework-Specific Pitfalls

Every framework gate you skip is a bet that nothing needed catching

Where Teams Get Stuck

Stale checklist

Treating a passed checklist as permanently done rather than re-running it after changes: a checklist reflects the spec at the moment it was generated, and later edits can silently invalidate it, so a checklist needs revisiting whenever the spec it was built from changes.

Ignoring analyze

Ignoring `/speckit.analyze` findings instead of returning to the step that owns them (specify or clarify for requirement problems, plan for design problems) lets small inconsistencies compound into much larger ones by the time implementation is done.

Documented Failure Modes

From OpenSpec's own docs

"Change not found," "No artifacts ready," and "Artifacts not generating properly" are named directly in OpenSpec's own troubleshooting guide. Thin project context in config.yaml is a common root cause, so the fix is often as simple as filling in more project context.

```
/opsx:ff
```

Fast-forward generates every artifact at once. Convenient, but it skips the incremental review that `/opsx:continue` gives you, an under-specification risk if nobody reads each artifact before the next is generated on top of it.

Module 8 at a Glance

	GitHub Spec Kit	OpenSpec
Common pitfall	Stale checklists or ignored analyze findings	Over-using /opsx:ff without review
Root cause	A review step was skipped	A review step was skipped
Documented by	Workflow reference (checklist, analyze)	OpenSpec's own troubleshooting guide

Checkpoint: What do Spec Kit's and OpenSpec's most common pitfalls have in common?

- Both trace back to a skipped review step, not a tooling failure.
- Both are documented, and both are avoidable once you know to look for them.
- Neither framework enforces the review, it enables it, the team still has to actually use it.

Putting It All Together

Modules 0–1

Foundations, and why spec-driven development over vibe coding

Modules 2–4

Discovery, governance, and a clarified spec through to a reviewable plan

Modules 5–6

Quality gates before code, human review after it

Module 7

Coordinating multiple agents and teams

Module 8

Recognizing the shortcuts that catch up with you

Key Takeaways

- Anti-patterns are rarely new mistakes; they're earlier shortcuts catching up with you later in the lifecycle.
- Every framework gate you skip is a bet that nothing needed catching, and that bet doesn't always pay off.
- Spec Kit's and OpenSpec's most common pitfalls trace to the same root cause: a skipped review step, not a tooling gap.
- The habits from this course don't prevent every mistake, they just make the mistakes visible early enough to fix cheaply. That's the whole premise, applied back to your own team's work starting now.